Assignment No 5

Aim: Apply the KNN algorithm for the classification.

Theory:

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for **classification and regression** problems. In classification, KNN predicts the class of a new data point by looking at the classes of its nearest neighboring data points.

KNN is called a **lazy learning algorithm** because it does not build an explicit model during the training phase. Instead, it stores the training data and performs calculations when a new data point needs to be classified.

For example, consider a dataset containing information about students such as **study hours, attendance, and marks**. KNN can classify a new student into categories such as **Pass** or **Fail** by comparing the student with similar students in the training dataset.

Working of KNN

The basic steps involved in KNN classification are:

1. **Choose the value of K:**
   K represents the number of nearest neighbors that will be considered for classification.

2. **Calculate distance:**
   The distance between the new data point and all training data points is calculated. The commonly used distance measure is **Euclidean distance**.

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

3. **Find K nearest neighbors:**
   The data points having the smallest distances from the new data point are selected.

4. **Perform majority voting:**
   The classes of the K nearest neighbors are considered, and the class that occurs most frequently is selected.

5. **Assign the class:**
   The new data point is assigned to the class receiving the highest number of votes.

Example

Suppose **K = 5** and the five nearest neighbors of a new data point have the following classes:

- Class A
- Class A
- Class B
- Class A
- Class B

Class A occurs **3 times**, while Class B occurs **2 times**.

Therefore, KNN classifies the new data point as **Class A**.

Selection of K

The value of **K** has an important effect on the performance of the algorithm.

- A **small K** can make the model sensitive to noise and outliers.
- A **large K** can make the model less sensitive to individual data points and may lead to underfitting.

An appropriate value of K is usually selected by testing different values and choosing the one that provides good performance on validation data.

Data Preprocessing

Since KNN is a **distance-based algorithm**, feature scaling is very important. If one feature has a much larger numerical range than another, it can dominate the distance calculation.

Common scaling techniques include:

- **Standardization**
- **Min-Max normalization**

Advantages of KNN

- Simple and easy to understand.
- Easy to implement.
- No complex training process.
- Can be used for both classification and regression.
- Works well with smaller datasets.
- Can handle multi-class classification.

Limitations of KNN

- Prediction can be slow for very large datasets.
- Requires appropriate selection of K.
- Sensitive to irrelevant features.
- Sensitive to the scale of features.
- Can be affected by noisy data and outliers.
- Requires storing the training dataset.

Applications of KNN

KNN is used in various applications such as:

- Image classification.
- Pattern recognition.
- Recommendation systems.
- Handwriting recognition.
- Medical diagnosis.
- Customer classification.
- Credit risk classification.
- Text classification.

Conclusion

K-Nearest Neighbors (KNN) is a simple and effective supervised learning algorithm that classifies a new data point based on the classes of its nearest neighbors. Feature scaling and proper selection of **K** are important for obtaining good classification performance.

Code:

```
import pandas as pd

from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
data = load_wine()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

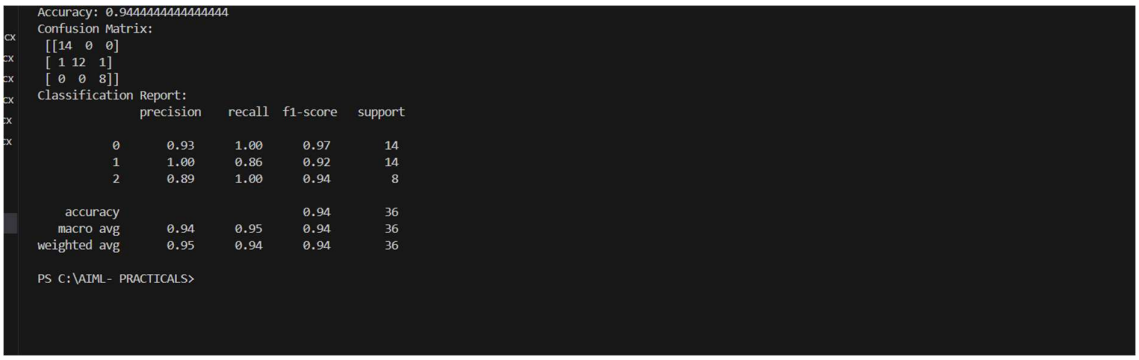
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

k = 5
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train_scaled, y_train)

y_pred = knn.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", conf_matrix)
print("Classification Report:\n", report)
```

Output:



```
Accuracy: 0.9444444444444444
Confusion Matrix:
 [[14  0  0]
 [ 1 12  1]
 [ 0  0  8]]
Classification Report:
              precision    recall  f1-score   support

           0       0.93      1.00      0.97        14
           1       1.00      0.86      0.92        14
           2       0.89      1.00      0.94         8

    accuracy                           0.94        36
   macro avg       0.94      0.95      0.94        36
weighted avg       0.95      0.94      0.94        36

PS C:\AIML- PRACTICALS>
```